Moebius Vue3 - Chat UI

Table of content

Getting started

First of all, Thank you so much for purchasing this template and for being our loyal customer. You are awesome! You are entitled to get free lifetime updates to this product and support from the css ninja team directly. **Moebius** is a product built by Css Ninja and Digisquad.

This documentation has been written to help you regarding each step of setup and customization. Please go through the documentation carefully to understand how this template is made and how to edit this properly. HTML CSS, and Vue JS framework knowledge is required to customize this template.

You are currently reading the **Moebius Vue3 1.0.0** documentation. The product uses:

- `vue3` Composition API
- Lightning-fast `vitejs` build & development tool
- `Typescript` out of the box, for large-scale JavaScript applications for any browser
- Typekit's `webfontloader` utility to load web fonts from any sources
- Latest `bulma` integration with `sass`
- Mocked HTTP Rest API responses to help you to create your own backend
- Production ready `docker` images based on bitnami
- `yarn` and `npm` support
- `eslint`, `stylelint` and `prettier` pre-configured

Support

If you have any trouble while editing this template or if you simply want to ask a question about something, feel free to contact us at support@cssndinja.io or to post your request on our support at cssninja.io

Prerequisites

1. A good code editor
   VSCode settings are preconfigured
2. A supported web browser (Chrome, Edge, Firefox, ...)
3. Nodejs LTS *(14.x)* installed
4. Typescript *(4.x)* installed

Dependencies installation

To setup the template and start installing project dependencies, run one of the following commands:

```
# using npm
npm install

# using yarn
yarn install
```

Run a development server

To start the development server, run one of the following commands:

```
# using npm
npm run dev

# using yarn
yarn dev
```

> Access the Moebius frontend in your browser at http://localhost:3000/

Moebius and Vue 3

Vue 3 introduces the Composition API wich is a set of additive, function-based APIs that allow flexible composition of component logic.

You may want to take a look of what changed since Vue 2 here:
https://v3.vuejs.org/guide/migration/introduction.html#overview

You still can still write components using old Option API (`data` / `computed` / `method` components fields).
Therefore, all Vue2 plugins/components should be compatible with Vue3.

Here is a great cheat sheet provided by the vue core team to help you get started with the Composition API:
https://www.vuemastery.com/pdf/Vue-3-Cheat-Sheet.pdf

Global structure

```
Moebius
|-- .vscode/
|-- nginx/
|-- public/
|     |-- api/
|     |-- img/
|
|-- src/
|     |-- assets/
|     |-- components/
|     |-- composition/
|     |-- layouts/
|     |-- models/
|     |-- pages/
|     |-- partials/
|     |-- App.vue
|     |-- main.ts
|     |-- router.ts
|     |-- shims-html5.d.ts
|     |-- shims-vue.d.ts
|
|-- .dockerignore
|-- .editorconfig
|-- .eslintrc.js
|-- .gitignore
|-- .prettierignore
|-- .prettierrc.js
|-- Dockerfile
|-- index.html
|-- package-lock.json
|-- package.json
|-- stylelint.config.js
|-- tsconfig.json
|-- vite.config.js
|-- yarn.lock
```

The directory structure can seem unusual for someone who is not familiar with build tools and javascript bundlers. We will break each part of it, so you understand how everything works.

Router, layouts and pages structure

```
Moebius
|-- src/
|     |-- layouts/
|          |-- AppLayout.vue
|          |-- AuthLayout.vue
|          |-- LandingLayout.vue
|          |-- WizardLayout.vue
|
|     |-- pages/
|          |-- auth/
|          |-- workspace/
|          |-- 404.vue
|          |-- index.vue
|
|     |-- router.ts
|
```

Moebius uses `vue-router` for Vue 3, you can find more information by visiting the corresponding documentation: https://next.router.vuejs.org/

Vue router allows to dynamically render a component as a page based on a provided url.
In order to do so, we have to bind each route to a component.

Page components are regular components, they live under the `pages/` folder. *Note: You can create your page files anywhere but we recommend that you do it inside the `pages/` folder as it helps keeping the project structured*

The routes are defined in the ./src/router.ts file.
Here is an example of registering a new page with a new layout

```
// file ./src/router.ts
import { RouteRecordRaw } from 'vue-router'

// lazyload component using "const Component = () => import()" syntax
const MyLayout = () => import('@src/layouts/MyLayout.vue')

const routes: RouteRecordRaw[] = [
  {
    // all routes declared as children will use MyLayout
    path: '/my-path',
    component: MyLayout,
    children: [
      {
        // match exact url: /my-path
        path: '',
        component: () => import('@src/pages/index.vue'),
      },
      {
        // match exact url: /my-path/my-page
        path: 'my-page',
        name: 'my-path-my-page', // you can set a name for your route here
        component: () => import('@src/pages/my-page.vue'),
      },
    ],
  },
]
```

To create a new layout, your component's template needs to have a `<RouterView />`, which gets replaced by your page component.
Note: You can create your layout files anywhere but we recommend that you do it inside the `layouts/` folder as it helps keeping the project structured

Components structure

```
Moebius
|-- src/
|     |-- components/
|          |-- common/
|          |-- landing/
|          |-- users/
|          |-- workspace/
|               |-- drawers/
|               |-- dropdown/
|               |-- messages/
|               |-- modals/
|               |-- panels/
|
```

The `components` folder holds all reusable elements. They consist in chunks of code that you can reuse accross your application: it can be a button, a navbar, a content section or whatever you want. You can create as many subfolders as you want to organize your components.

Here is an example of a component:

```
<script lang="ts">
// file: ./src/components/MyComponent.vue
import { defineComponent } from 'vue'

// You can load any other components ('@src/' is an alias to '<base>/src' folder)
import OtherComponent from '@src/components/OtherComponent.vue'

const MyComponent = defineComponent({
  name: 'MyComponent',
  components: {
    OtherComponent,
  },
  setup() {
    // MyComponent Composition API

    return {}
  },
})

export default MyComponent
</script>

<template>
  <!-- MyComponent Template -->

  <OtherComponent />
</template>
```

> Tip for VSCode users:
> start typing `<script` in an empty `.vue` file to trigger a snippet suggestion.

Naming a component:

Some components names begin with `The`, like `TheConversation.vue`. Those are inteded to have a single active instance.
Also when referring to components inside the template, consider using `CamelCase` insted of `pascal-case` notation.

> Tip: You can find all recommandations and best practices in the Vue3 style guide
> https://v3.vuejs.org/style-guide

The Composition API

```
Moebius
|-- src/
|     |-- composition/
|          |-- state/
|               |-- ui/
|               |-- user/
|               |-- workspace/
|          |-- use/
|
```

Since we now have the powerful Composition API, we do not need to use `vuex` anymore to share state between components.
Here is an example using vuex composition api
Instead we register variables in separate files using the `ref` or `reactive` methods from the new `vue` package. Those variables will be reactive across all your project. You can see examples of it in `./src/composition/state`

Also, to reduce component size and allow easy code reusability, code logic is split into **Composable** functions. Those functions create a scoped state (usefull for a reusable dropdown, for instance). You can see examples of it in `./src/composition/use`

> Tip: keep the cheat sheet open!
> https://www.vuemastery.com/pdf/Vue-3-Cheat-Sheet.pdf

> Advanced: Why the Composition API was introduced in Vue https://v3.vuejs.org/guide/composition-api-introduction.html

Data API fixtures

```
Moebius
|-- public/
|     |-- api/
|          |-- auth/
|               |-- login.json
|               |-- register.json
|          |-- conversations/
|               |-- conversation-x.json
|               |-- users.json
|          |-- users.json
```

You probably already noticed that there are `json` files located in the `public/api` folder.
Those files are requested by our state Composition API to simulate calls to a real world API.

The structure is arbitrary and probably won't reflect what you will need to implement for your backend. This fake API is simply here to give you examples of on how to request remote data using the Composition API (we are using the `axios` package, which is one of the most common for HTTP REST requests).

Vite, Rollup and Typescript

```
Moebius
|-- tsconfig.json
|-- vite.config.js
```

Moebius uses `Vite`, which is a web development build tool that supports:

- a fast development environment with hot reload *(using `native javascript modules`)*
- building an optimized version for production *(using `rollup` and `tree-shaking`)*
- native support of typescript

Under the hood, when runing `npm run dev`, it runs `vite`.

To learn more about this awesome tool made by the vuejs core team, check the Vite documentation. You will learn how to:

- Add postcss pre-processors
- Integrate JSX
- Implement fancy Web Assembly code
- Add new asset path aliases
- And much more

Typescript is just an extention of Javascript, If you are new to it, don't be afraid because all valid JavaScript code is also TypeScript code. *(TypeScript documentation)*
The main advatages of using Typescript are:

- Validates your code ahead of time
- Provides auto-completion
- Supports complex Type checking

Keep it clean with linters

```
Moebius
|-- .vscode/
|     |-- extensions.json
|     |-- settings.json
|
|-- .editorconfig
|-- .eslintrc.js
|-- .prettierignore
|-- .prettierrc.js
|-- stylelint.config.js
```

Because we love clean code, we have configured 3 linters which have their own purpose:

- `eslint` : prevent code quality concerns (no unused vars, ...)
- `stylelint` : prevent css quality concerns (no invalid colors, ...)
- `prettier` : handles formatting rules (max line lenght, ...)

You can check the quality of your code by running

```
# using npm
npm run lint

# using yarn
yarn lint
```

Linters can fix a lot of issues all by themselves. To do so, try runing

```
# using npm
npm run lint:fix

# using yarn
yarn lint:fix
```

> Tip for VSCode users:
> Install recommended extensions, linting will then occur each time files are saved!

Customizing Moebius

Scss structure

```
Moebius
|-- src/
|     |-- assets/
|          |-- scss
|               |-- abstracts
|               |-- base
|               |-- components
|               |-- layout
|               |-- pages
|               |-- main.scss
|     |-- main.ts
|-- index.html
```

Moebius relies on the powerful Sass features and a modular structure, letting you handle complex styles in a breeze. You need to import all the SCSS partials into your core file. This is how scss files are organized. Partial SCSS file names always start with an underscore like this: `_button.scss` . They act as chunks of code that you can import only if you need them. Of course some of them are mandatory for the project to work. Moebius is a UI kit in wich each SCSS file serves a different purpose:

- **abstracts**: holds all files related to SCSS variables, mixins, default global styles and other typography settings.
- **components**: holds all files related to Moebius Components. Each component has its own file.
- **layout**: holds all basic and mandatory layout files. The project will break if you ommit one of those files.
- **pages**: holds all the specific styles for each demo or group of demos.

Import Scss in Vue3

In order to load stylesheets into our application, we simply need to import `css` , `sass` or `scss` files in the `main.ts` file

```
// file: ./src/main.ts

// ...

import '@fortawesome/fontawesome-free/scss/fontawesome.scss'
import '@fortawesome/fontawesome-free/scss/regular.scss'
import '@fortawesome/fontawesome-free/scss/solid.scss'
import '@fortawesome/fontawesome-free/scss/brands.scss'
import 'lightgallery.js/dist/css/lightgallery.css'
import 'vue-toastification/dist/index.css'

import '@src/assets/scss/main.scss'

// ...
```

> all imported files here are to be converted in `css` , optimized using `postcss` , and injected automatically in `index.html` at build and run time.

Bulma Integration

Bulma is fully integrated with Moebius. This means that when you change the `$primary` Moebius color variable and any Bulma related variable to take precedence over it. The variable changes you make will be applied to all native Bulma elements.

Native Dark Mode

Moebius comes with a native Dark mode. This means that all components are prestyled for dark mode. You don't have to worry about it, when you turn it on, the colors change seamlessly. Dark mode styling is made through a global `.is-dark` class added to the page `<body>` element. Dark mode styling is centralized into a single file, `_dark.scss` . You can remove this file from the imports if you do not want to use Dark mode in your project. Dark mode is toggled on the body by a simple javascript function. In a real world implementation, the body would have to be rendered by the server with the proper class before being served to the client, based on the user selection.

Lazyloading Scss Pages

```
Moebius
|-- src/
|     |-- assets/
|          |-- scss
|               |-- pages/
|                    |-- _landing.scss
|                    |-- _login.scss
|                    |-- _wizard.scss
|               |-- main.scss
```

The pages folder holds the template pages that are not required by the main chat application.

They are not added by default to `main.scss` , instead we lazyload them in layouts:

```
<!-- file ./src/layouts/WizardLayout.vue -->
<script lang="ts">
  // ...
</script>

<template>
  <!-- ... -->
</template>

<style lang="scss">
  /* files imported in components will be loaded only once they are needed */
  @import '../assets/scss/pages/wizard';
</style>
```

Going further

Build for production

```
# using npm
npm run build

# using yarn
yarn build
```

> Built files are located in ./dist folder
> You can deploy your website on any http server like Apache, Nginx or `http-server` package from npm
> You can also use a CDN like Github pages, Netlify, AWS Cloudfront, ...
> **Moebius can not be opened from the local file system (using `file://` protocol)**

You can preview quickly your production version with `http-server`

```
npx http-server ./dist
```

> Now you can visit http://localhost:8080 to view your server

Using docker

This project includes a Dockerfile which first builds Moebius for production, and then creates a tiny image with only built files, served by Nginx

To build your image, run the following command:

```
docker build --tag moebius:1.0 .
```

To preview your image, run the following command:

```
docker run --publish 8080:8080 --rm moebius:1.0
```

> Access the Moebius frontend at http://localhost:8080

To run your image, run the following command:

```
docker run --publish 80:8080 --detach --restart always --name my-moebius-app moebius:1.0
```

> *note: you might need root/sudo access to bind port 80 to the container*

Build your own backend

To bring your chat alive you will need to create a backend for user authentification, message and room persistance, etc ...
We have implemented samples with a fake REST HTTP API to keep the project as simple and understainable as possible.

You can take a look at projects such strapi or kuzzle, wich are open-source backend, that can be nicely used with Moebius *(firebase can be a good choice too)* !

Customer support

Please remember you have purchased a very affordable theme and you did not pay for a full-time web design agency. We will help with your issues, but these requests will be put on a relevant priority, regarding their nature. Support is provided for your comfort and for a best possible experience, so please be patient, polite and respectful, as we are towards you.

Support includes

- Responding to questions or problems regarding the item and its features
- Fixing bugs and reported issues
- Providing updates

Support does not include

- Customization and installation services
- Support for third party software and plug-ins

Before Support

- Make sure your question is a valid Theme Issue and not a customization request
- Make sure you have read through the documentation and any ressources before asking support on how to accomplish a task
- Make sure to double check the theme FAQs.
- If you have customized your theme and now have an issue, back-track to make sure you didn't make a mistake. If you have made changes and can't find the issue, please provide us with your changelog.
- Almost 80% of the time we find that the solution to people's issues can be solved with a simple "Google Search". You might want to try that before seeking support. You might be able to fix the issue yourself much quicker than we can respond to your request.
- Make sure to state the name of the theme you are having issues with when requesting support.

Changelog

You can find the version history CHANGELOG.md file inside the moebius-theme.zip folder or you can check the changelog on the theme's sale page.

Once again, thank you so much for purchasing this theme. As I said at the beginning, we'd be glad to help you if you have any questions related to this theme. No guarantees, but we'll do our best to assist and support you. If you have a more general question relating to Moebius Vue3, you might consider opening a ticket and ask your question in the Css Ninja support portal.